

Tarefa 6: Escopo de Variáveis e Regiões Críticas em OpenMP

Israel Soares de Castro Filho

Matrícula: 20250070780

1 Introdução

Este trabalho tem como objetivo estudar, na prática, os conceitos de escopo de variáveis e regiões críticas em programação paralela com OpenMP, usando como estudo de caso a estimativa estocástica do número π pelo método de Monte Carlo. A ideia do método é gerar pontos aleatórios uniformemente distribuídos no quadrado $[-1, 1] \times [-1, 1]$ e contar a fração deles que cai dentro do círculo unitário inscrito; como a razão entre as áreas do círculo e do quadrado é $\pi/4$, tem-se

$$\pi \approx 4 \cdot \frac{n_{\text{dentro}}}{n_{\text{total}}}.$$

Diferentemente de métodos determinísticos (como a série de Gregory–Leibniz), esse problema é interessante para o estudo de paralelismo porque a etapa de contagem envolve uma variável compartilhada (`dentro`) atualizada por todas as iterações, o que expõe de forma direta o problema de *condição de corrida* quando o laço é paralelizado ingenuamente. O objetivo específico da tarefa é:

1. Implementar a versão sequencial e uma versão paralela ingênua com `#pragma omp parallel for`;
2. Mostrar e explicar o resultado incorreto produzido pela condição de corrida;
3. Corrigir o problema com `#pragma omp critical`;
4. Reestruturar o código separando `#pragma omp parallel` e `#pragma omp for`, aplicando as cláusulas `private`, `firstprivate`, `lastprivate` e `shared`;
5. Discutir o papel de `default(none)` na clareza do escopo em programas complexos.

2 Metodologia

O código foi implementado em C (arquivo `tarefa6.c`, Apêndice A) e paralelizado com OpenMP. Foram implementadas quatro versões da estimativa de π , todas utilizando o mesmo gerador de números pseudoaleatórios *thread-safe* (Xorshift32, com estado passado por referência e nunca inicializado em zero, para evitar a degeneração conhecida desse gerador):

- **Ingênua** (`pi_paralelo_com_erro`): paraleliza o laço com `#pragma omp parallel` for simples, incrementando diretamente uma variável `long` dentro compartilhada e chamando `rand()` (função com estado global, não *thread-safe*) dentro da região paralela. Serve para evidenciar a condição de corrida.
- **Critical** (`pi_paralelo_critical`): reestruturada com `#pragma omp parallel` seguido de `#pragma omp for`, com a semente de cada thread inicializada uma única vez (`private` implícita por ser declarada dentro do bloco paralelo) e o incremento de dentro protegido por `#pragma omp critical` a cada ponto dentro do círculo.
- **Otimizada** (`pi_paralelo_otimizado`): cada thread acumula em um contador local `dentro_local` (privado) durante todo o laço `#pragma omp for` e só executa uma seção crítica *uma vez*, ao final, para somar seu resultado parcial na variável compartilhada `dentro_total`.
- **Reduction** (`pi_paralelo_reduction`): usa a cláusula `reduction(+:dentro)` no próprio `#pragma omp parallel`, deixando o runtime OpenMP responsável por criar cópias privadas por thread e combiná-las automaticamente ao final da região.

Além disso, a função `demonstrar_clausulas` isola, em um laço simples e independente do problema de Monte Carlo, o efeito de cada cláusula de escopo (`private`, `firstprivate`, `lastprivate`, `shared`) sobre quatro variáveis de controle (`a`, `b`, `c`, `d`), permitindo observar diretamente o valor de cada uma antes e depois da região paralela.

Todas as regiões paralelas usam `default(none)`, exigindo que cada variável usada seja classificada explicitamente. A medição de tempo é feita com `omp_get_wtime()` (tempo de relógio, *wall-clock*).

O programa foi compilado com:

```
$ gcc -std=c11 -fopenmp -O2 -Wall -o tarefa6 tarefa6.c -lm
```

e executado em uma máquina com `omp_get_max_threads() = 20`, testando amostras de 10^3 a 10^7 pontos.

3 Resultados

A demonstração das cláusulas de escopo produziu a seguinte saída:

```
Antes do laço:  a=10  b=10  c=10  d=0
Depois do laço: a=10 (indefinido/nao usar)
                  b=10 (inalterado fora, era firstprivate)
                  c=7  (deve ser 7, ultima iteracao)
                  d=8  (deve ser 8, soma protegida)
```

Quanto à estimativa de π ($\pi_{\text{real}} = 3,1415926536$), a versão ingênua produziu valores cada vez mais distantes do correto à medida que o número de pontos cresce: 2,956 (10^3 pontos), 0,878 (10^4), 0,840 (10^5), 0,368 (10^6) e 0,263 (10^7). Já as versões *Critical*, *Otimizada* e *Reduction* produziram, em cada tamanho de amostra, exatamente o mesmo valor entre si — 3,156; 3,1448; 3,14704; 3,138704; e 3,1408096, respectivamente — todos com erro absoluto decrescente e da ordem de 10^{-3} já a partir de 10^6 pontos, como esperado de uma implementação correta.

A Figura 1 apresenta o tempo de execução de cada versão em função do número de pontos, em escala log-log. Para 10^3 e 10^4 pontos, o tempo medido ficou em 0,000s para praticamente todas as versões (a exceção foi um único registro de 0,001s em *Reduction* com 10^3 pontos, atribuível a ruído de inicialização) — ou seja, abaixo da resolução útil do cronômetro nessas amostras pequenas, sem informação de desempenho a extrair. Por isso, a figura mostra apenas $n = 10^5, 10^6, 10^7$, faixa em que as diferenças entre versões já são mensuráveis; os dois pontos de *Reduction* iguais a zero nessa faixa (10^5 e 10^6) foram marcados com um “×” e posicionados em um piso visual, apenas para não quebrar a escala logarítmica.

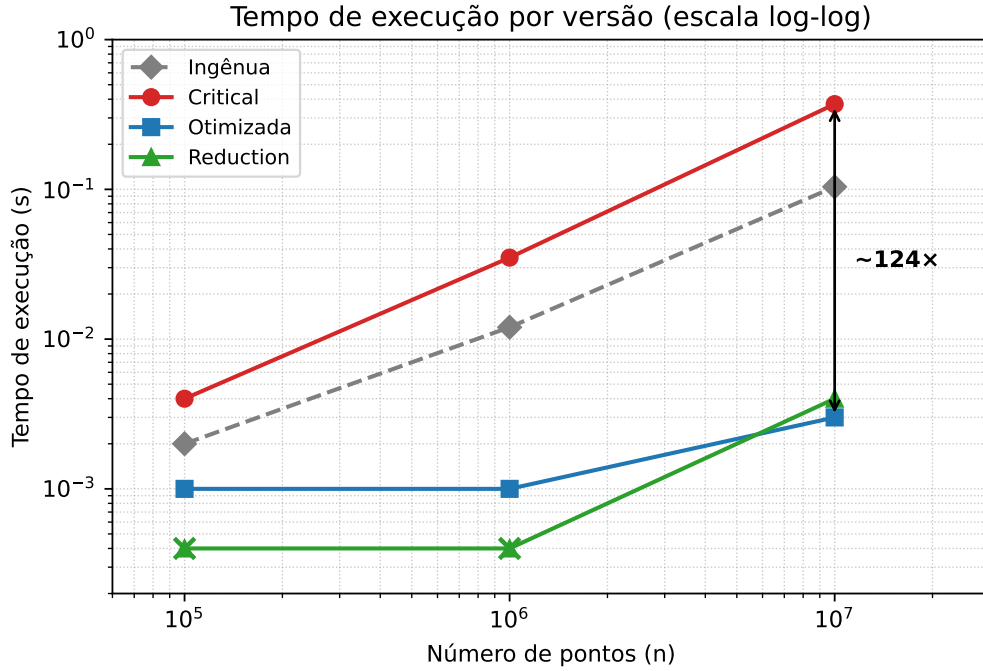


Figure 1: Tempo de execução por versão em função do número de pontos (escala log-log). Marcadores “×” indicam tempo medido igual a 0,000s (abaixo da resolução do cronômetro), reposicionados em um piso visual apenas para permitir a escala log.

4 Análise

O resultado incorreto da versão ingênua. Os valores de π apresentados acima mostram que a versão ingênua não apenas erra, como o erro *cresce sistematicamente* com o número de pontos, em vez de apenas oscilar em torno de π como seria esperado de um simples ruído estatístico: o valor estimado cai de 2,956 (com 10^3 pontos) até 0,263 (com 10^7 pontos). Isso tem duas causas combinadas:

1. **Condição de corrida em dentro++.** A operação é, na verdade, três passos (ler, incrementar, escrever) que não são atômicos. Com 20 threads disputando a mesma variável, várias leituras ocorrem antes de qualquer escrita ser propagada, de modo que incrementos são perdidos. Quanto mais pontos e mais threads simultâneas, maior a frequência de colisões, então a fração de incrementos perdidos tende a

aumentar com n — o que explica o viés crescente observado, e não apenas uma dispersão aleatória em torno do valor correto.

2. **rand() não é *thread-safe*.** A função mantém estado interno global; chamadas concorrentes de 20 threads corrompem esse estado, de modo que os valores gerados deixam de ser uma amostra i.i.d. válida, o que também contribui para o viés sistemático, e não apenas a perda de incrementos.

Correção com `critical`. Ao proteger `dentro++` com `#pragma omp critical` e usar um gerador próprio por thread (Xorshift32 com semente independente), o resultado passa a convergir corretamente para π (erro da ordem de 10^{-3} já em 10^6 – 10^7 pontos). O custo, porém, é alto: como cerca de 78% dos pontos caem dentro do círculo, quase toda iteração do laço precisa entrar na seção crítica, serializando essa fração do trabalho. Isso é visível na Figura 1: em 10^7 pontos, a versão *Critical* leva 0,371 s, cerca de **124 vezes mais lenta** que a versão *Otimizada* (0,003 s).

Reestruturação com `parallel + for` e as cláusulas de escopo. As versões *Otimizada* e *Reduction* evitam o gargalo movendo a seção crítica para *fora* do laço: cada thread acumula em uma variável local (privada, por ser declarada dentro do bloco `#pragma omp parallel`) e só sincroniza uma vez, ao final. O resultado nos tempos é dramático: ambas ficam na faixa de milissegundos mesmo em 10^7 pontos, contra centenas de milissegundos da versão *Critical*. A saída de `demonstrar_clausulas` confirma o comportamento esperado de cada cláusula:

- **private(a):** cada thread recebe uma cópia isolada de `a`; como o valor não é propagado de volta, `a` permanece com o valor anterior à região paralela (10) fora do laço, mesmo tendo sido reescrito internamente.
- **firstprivate(b):** como `private`, mas a cópia de cada thread é inicializada com o valor de `b` anterior ao laço (10); o valor externo também permanece inalterado após a região, pois não há cópia de volta.
- **lastprivate(c):** ao final da região, `c` recebe o valor calculado pela iteração sequencialmente *última* do laço ($i = 7$), o que foi confirmado pela saída (`c=7`).
- **shared(d):** todas as threads acessam a mesma posição de memória; sem proteção (aqui, `#pragma omp atomic`), o incremento estaria sujeito à mesma condição de corrida da versão ingênua. Com a proteção, o resultado final foi o esperado (`d=8`, uma unidade por iteração).

Papel de `default(none)`. Sem essa cláusula, qualquer variável referenciada dentro de uma região paralela e não classificada explicitamente assume por padrão o escopo `shared` — que foi exatamente a causa raiz do bug da versão ingênua. Com `default(none)`, o compilador passa a exigir que *toda* variável usada na região seja explicitamente declarada como `private`, `firstprivate`, `lastprivate` ou `shared`; se alguma for esquecida, o código sequer compila.

5 Conclusão

O experimento evidenciou de forma concreta como a ausência de controle de escopo em OpenMP leva a condições de corrida com impacto crescente sobre a precisão do resultado, e não apenas a ruído aleatório. A correção ingênua com `#pragma omp critical` resolve a corretude, mas ao custo de um gargalo de sincronização que pode tornar o código paralelo mais lento que o sequencial quando a seção crítica é executada com muita frequência.

A reestruturação com acumuladores privados por thread (seja manualmente, com `private` e uma única seção crítica de junção, seja de forma idiomática, com `reduction`) elimina esse gargalo e recupera o ganho de desempenho esperado do paralelismo.

Por fim, as cláusulas `private`, `firstprivate`, `lastprivate` e `shared` definem precisamente o que cada thread vê e o que é propagado de volta ao término da região, e `default(none)` torna essas decisões explícitas e verificáveis em tempo de compilação — uma prática recomendável especialmente à medida que o programa cresce em complexidade.

A Código-Fonte

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <time.h>
5 #include <omp.h>
6 #include <limits.h>
7
8 #define PI_REAL 3.14159265358979323846
9
10 /* Gerador Xorshift32 */
11 static inline unsigned int xorshift32(unsigned int *state)
12 {
13     unsigned int x = *state;
14
15     x ^= x << 13;
16     x ^= x >> 17;
17     x ^= x << 5;
18
19     *state = x;
20     return x;
21 }
22
23 /* Geracao de double aleatorio */
24 static inline double random_double(unsigned int *state)
25 {
26     return (double)xorshift32(state) / (double)UINT_MAX;
27 }
28
29 /* Cria uma semente diferente por thread e NUNCA igual a zero */
30 static inline unsigned int semente_da_thread(void)
31 {
32     unsigned int base = (unsigned int)time(NULL) ^
33         ((unsigned int)omp_get_thread_num() * 2654435761u);
34     return base | 1u;
35 }
36
37 /* =====
38  Estimativa de pi por Monte Carlo:
39  gera pontos aleatorios em [-1,1] x [-1,1] e verifica quantos caem
40  dentro do circulo unitario. A razao (pontos_dentro / total) * 4
41  aproxima pi (area do circulo / area do quadrado = pi/4).
42  ===== */
43
44 /* ----- Versao sequencial (referencia, sem OpenMP) ----- */
45 double pi_sequencial(long n_pontos, unsigned int seed) {
46     seed |= 1u; /* garante estado inicial != 0 */
47     long dentro = 0;
48     for (long i = 0; i < n_pontos; i++) {
49         double x = random_double(&seed) * 2.0 - 1.0;
50         double y = random_double(&seed) * 2.0 - 1.0;
51         if (x * x + y * y <= 1.0) dentro++;
52     }
53     return 4.0 * (double)dentro / (double)n_pontos;
54 }
55
56 /* ----- Versao 1: paralela -> CONDICAO DE CORRIDA (intencional) ----- */
```

```

57 double pi_paralelo_com_erro(long n_pontos) {
58     long dentro = 0; /* variavel compartilhada por padrao (default(shared)) */
59
60     #pragma omp parallel for
61     for (long i = 0; i < n_pontos; i++) {
62         /* rand() usa estado global interno: nao e thread-safe de proposito,
63         para preservar o efeito didatico da condicao de corrida */
64         double x = (double)rand() / RAND_MAX * 2.0 - 1.0;
65         double y = (double)rand() / RAND_MAX * 2.0 - 1.0;
66         if (x * x + y * y <= 1.0) {
67             dentro++; /* leitura + incremento + escrita: NAO e atomico */
68         }
69     }
70     return 4.0 * (double)dentro / (double)n_pontos;
71 }
72
73 /* ----- Versao 2: corrigida com #pragma omp critical ----- */
74 double pi_paralelo_critical(long n_pontos) {
75     long dentro = 0;
76
77     #pragma omp parallel default(none) shared(n_pontos, dentro)
78     {
79         unsigned int seed = semente_da_thread(); /* uma vez por thread */
80
81         #pragma omp for
82         for (long i = 0; i < n_pontos; i++) {
83             double x = random_double(&seed) * 2.0 - 1.0;
84             double y = random_double(&seed) * 2.0 - 1.0;
85             if (x * x + y * y <= 1.0) {
86                 #pragma omp critical
87                 dentro++; /* so uma thread por vez executa o incremento */
88             }
89         }
90     }
91     return 4.0 * (double)dentro / (double)n_pontos;
92 }
93
94 /* ----- Versao 3: reestruturada com omp parallel + omp for ----- */
95 double pi_paralelo_otimizado(long n_pontos) {
96     long dentro_total = 0;
97
98     #pragma omp parallel default(none) shared(n_pontos, dentro_total)
99     {
100         long dentro_local = 0; /* privada por ser local ao bloco paralelo */
101         unsigned int seed = semente_da_thread();
102
103         #pragma omp for
104         for (long i = 0; i < n_pontos; i++) {
105             double x = random_double(&seed) * 2.0 - 1.0;
106             double y = random_double(&seed) * 2.0 - 1.0;
107             if (x * x + y * y <= 1.0) dentro_local++;
108         }
109
110         #pragma omp critical
111         dentro_total += dentro_local; /* 1 secao critica por THREAD, nao por iteracao */
112     }
113 }

```

```

114     return 4.0 * (double)dentro_total / (double)n_pontos;
115 }
116
117 /* ----- Versao 4: abordagem idiomatica do OpenMP -> reduction ----- */
118 double pi_paralelo_reduction(long n_pontos) {
119     long dentro = 0;
120
121     #pragma omp parallel default(none) shared(n_pontos) reduction(+:dentro)
122     {
123         unsigned int seed = semente_da_thread();
124
125         #pragma omp for
126         for (long i = 0; i < n_pontos; i++) {
127             double x = random_double(&seed) * 2.0 - 1.0;
128             double y = random_double(&seed) * 2.0 - 1.0;
129             if (x * x + y * y <= 1.0) dentro++;
130         }
131     }
132     return 4.0 * (double)dentro / (double)n_pontos;
133 }
134
135 /* =====
136 Demonstracao didatica isolada de private / firstprivate / lastprivate
137 / shared, para observar o efeito de cada clausula separadamente.
138 ===== */
139 void demonstrar_clausulas(void) {
140     int a = 10, b = 10, c = 10, d = 0;
141
142     printf("Antes do laço: a=%d b=%d c=%d d=%d\n", a, b, c, d);
143
144     #pragma omp parallel for default(none) private(a) firstprivate(b) lastprivate(c)
145     shared(d)
146     for (int i = 0; i < 8; i++) {
147         a = i * 10;      /* copia PRIVADA por thread; começa com valor indefinido */
148         b += i;          /* copia privada, mas INICIALIZADA com o valor de fora (10) */
149         c = i;           /* ao sair do laço, 'c' externo recebe o valor calculado */
150                         /* pela iteracao logicamente ULTIMA (i = 7) */
151         #pragma omp atomic
152         d += 1;          /* variavel COMPARTILHADA; precisa de atomic/critical */
153     }
154
155     printf("Depois do laço: a=%d (indefinido/nao usar) "
156           "b=%d (inalterado fora, era firstprivate) "
157           "c=%d (deve ser 7, ultima iteracao) "
158           "d=%d (deve ser 8, soma protegida)\n\n", a, b, c, d);
159 }
160
161 int main(void) {
162     long tamanhos[] = {1000, 10000, 100000, 1000000, 10000000};
163     int n = sizeof(tamanhos) / sizeof(tamanhos[0]);
164
165     double t0, t1;
166     double tempo_erro, tempo_critical, tempo_otimizado, tempo_reduction;
167
168     srand((unsigned int)time(NULL));
169
170     printf("Threads disponiveis (omp_get_max_threads): %d\n\n", omp_get_max_threads());

```



```

171 demonstrar_clausulas();
172
173 printf("PI_REAL = %.10f\n\n", PI_REAL);
174
175 for (int idx = 0; idx < n; idx++) {
176     long k = tamanhos[idx];
177
178     /* ----- Versao Ingenua ----- */
179     t0 = omp_get_wtime();
180     double pi1 = pi_paralelo_com_erro(k);
181     t1 = omp_get_wtime();
182     tempo_erro = t1 - t0;
183
184     /* ----- Versao Critical ----- */
185     t0 = omp_get_wtime();
186     double pi2 = pi_paralelo_critical(k);
187     t1 = omp_get_wtime();
188     tempo_critical = t1 - t0;
189
190     /* ----- Versao Otimizada ----- */
191     t0 = omp_get_wtime();
192     double pi3 = pi_paralelo_otimizado(k);
193     t1 = omp_get_wtime();
194     tempo_otimizado = t1 - t0;
195
196     /* ----- Versao Reduction ----- */
197     t0 = omp_get_wtime();
198     double pi4 = pi_paralelo_reduction(k);
199     t1 = omp_get_wtime();
200     tempo_reduction = t1 - t0;
201
202     /* ----- Impressao dos resultados ----- */
203     printf("\n===== \n");
204     printf("Quantidade de Pontos: %ld\n", k);
205     printf("===== \n");
206     printf("%-15s %-15s %-12s\n", "Versao", "Pi Estimado", "Tempo (s)");
207     printf("----- \n");
208     printf("%-15s %-15.8f %-12.6f\n", "Ingenua", pi1, tempo_erro);
209     printf("%-15s %-15.8f %-12.6f\n", "Critical", pi2, tempo_critical);
210     printf("%-15s %-15.8f %-12.6f\n", "Otimizada", pi3, tempo_otimizado);
211     printf("%-15s %-15.8f %-12.6f\n", "Reduction", pi4, tempo_reduction);
212 }
213
214 return 0;
215 }

```

Código 1: Código-fonte completo (tarefa6.c)